Analytical Report — F1 Podium Prediction (GUC Milestone 1)

Course: Introduction to Artificial Intelligence — Milestone 1 (F1 Podium Prediction)

Team:

Name	Student ID	Tutorial
Youssef Ahmed	58-4250	T-27
Omar Eldafrawy	58-5371	T-11
Ismail Almoatazbellah	58-4720	T-16
Mohamed Elayadi	43-2918	T-22

GitHub: https://github.com/CrimeWizard/f1-podium-prediction
Kaggle notebook: https://www.kaggle.com/code/crimewizard/acl-ms1
Dataset: Ergast F1 CSVs (Kaggle: rohanrao/formula-1-world-championship-1950-2020)

---

1. Introduction

Task. For each driver in each race (grain: `raceId`, `driverId`), predict whether the driver finishes **on the podium** (finishing position `positionOrder` in {1, 2, 3}).

Prediction cutoff. Features use only information available **after qualifying and before the race** (grid, qualifying times/position, championship standings entering the race, recent form from **prior** races). We do not use post-race fields (race points, status, pit stops, lap times, or standings updated by the current race) as model inputs.

Evaluation. Primary metric: **PR-AUC** (podium is a minority class). Secondary: ROC-AUC and F1 at a probability threshold tuned on **validation** only. Train/validation/test is a **temporal split**: train seasons <= 2019, validation 2020–2021, test >= 2022.

---

2. EDA and cleaning

Figure provenance. The corrected Q2 figure is copied from the public Kaggle notebook output (`crimewizard/acl-ms1`, saved version 2, kernel version `360186961`). The remaining report figures are from its earlier successful Kaggle run (kernel version `359399439`).

EDA before cleaning. We inspected raw `positionOrder` in `results` to confirm finish positions are usable for the target. We plotted mean qualifying rows per race by year; the `qualifying` table is empty for early decades and becomes dense from the mid-1990s onward, so quali features are often missing for old seasons.

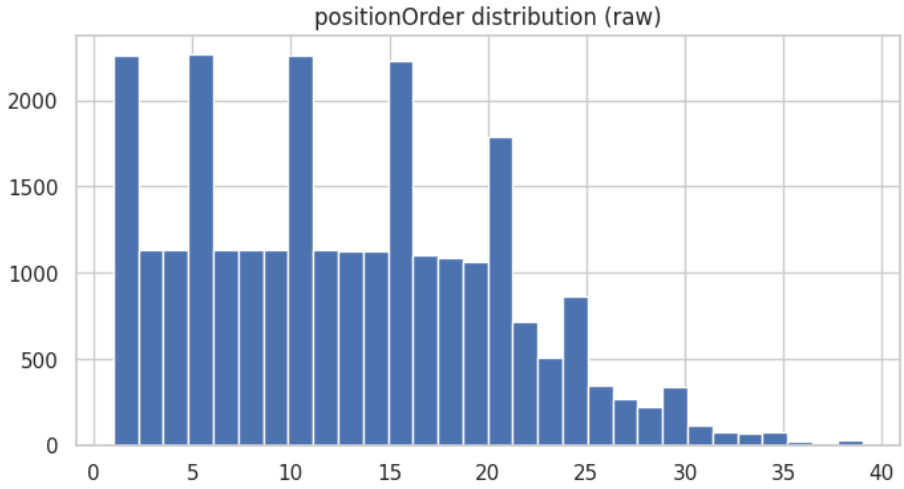


Figure 1: Raw finishing-position distribution

Cleaning. Missing sentinels (`\N`) are parsed as NA. We enforce **one row per (`raceId`, `driverId`)**, keeping the best `positionOrder` when shared-drive duplicates exist. **Indianapolis 500** championship rounds (1950–1960) are excluded from the main modeling base as non-comparable ovals. Primary and foreign key audits are reported in notebook section 2.

EDA after cleaning. We confirm podium rate by year on the cleaned table (notebook section 3).

---

3. Data-engineering questions

Q1 — Front row to podium by circuit. Front row = **grid** positions 1–2. We compare podium rates by circuit, split pre-2014 vs from-2014 (hybrid era). See notebook section 4 and figure `de_q1_front_row_podium.png`.

Q2 — Home race advantage. We compare podium rate for **home** vs **away** drivers within **grid bins**, so better qualifying is not confounded with home effect. See `de_q2_home_advantage.png`.

Q3 — Mechanical retirement by constructor. Ergast status strings are mapped to categories (`src/status_mapping.py`). Mechanical DNFs are counted

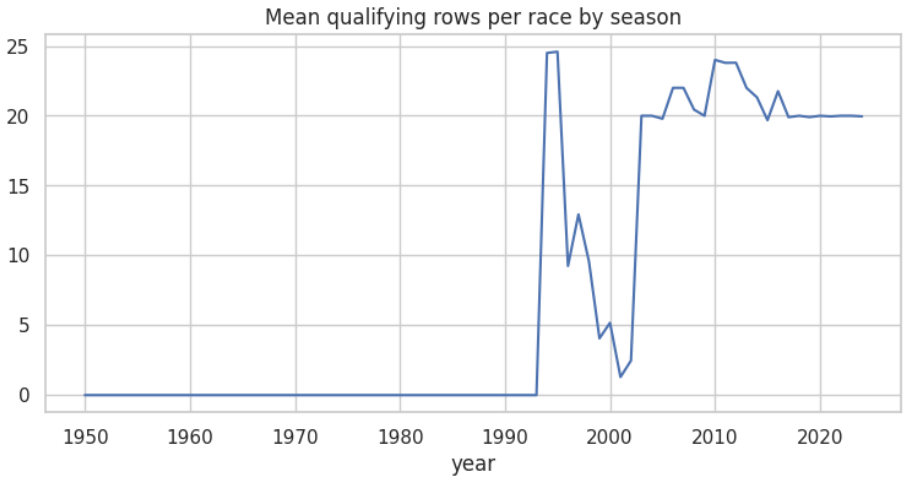


Figure 2: Qualifying coverage by year

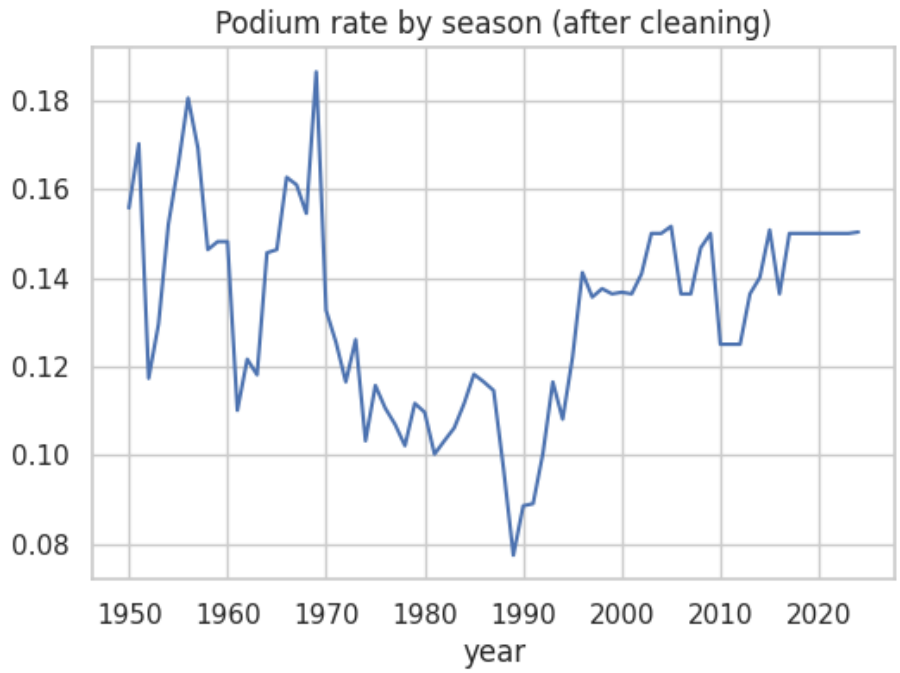


Figure 3: Podium rate by season

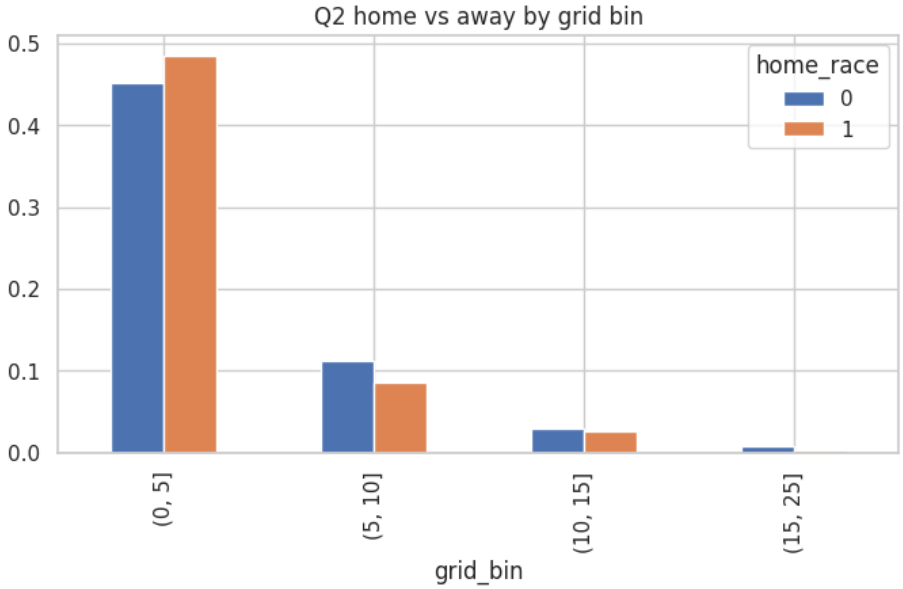


Figure 4: Home race advantage by grid bin

per constructor for grid > 0 starts, comparing 2014–2021 vs 2022–2024. Status category counts and tables are in notebook section 4.

---

4. Features (leakage-safe)

Feature	Meaning
`grid`	Starting position on race day
`quali_position`, `q1_sec`, `q2_sec`, `q3_sec`	Qualifying result (sparse before ~1995 in data)
`points_before`, `position_before`	Driver championship standing **before** this race
`constructor_points_before`, `constructor_position_before`	Constructor standing before race
`podium_rate_prev`	Share of podiums in prior races (rolling, excludes current race)

IDs (`raceId`, `driverId`, etc.) and `year` are not used as model inputs.

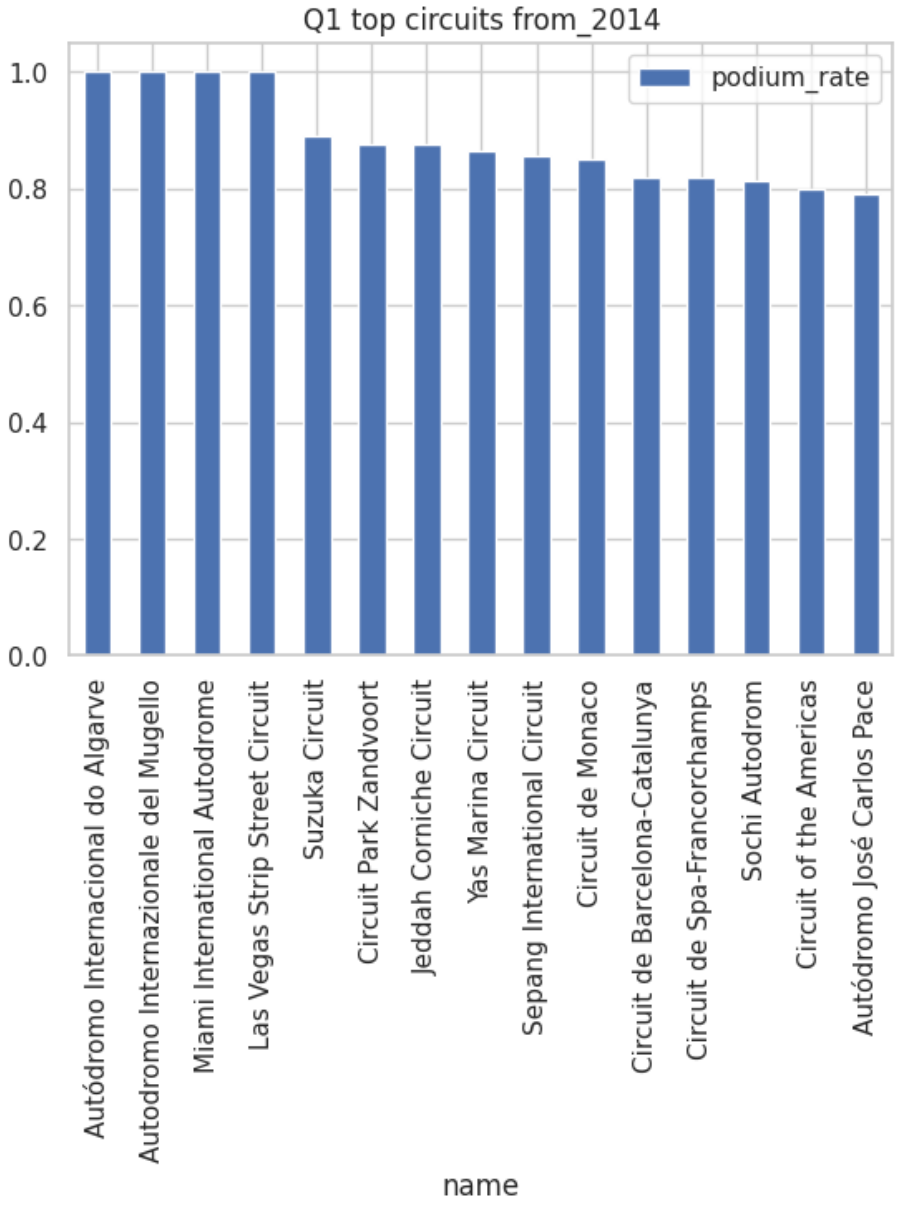


Figure 5: Front-row podium rates by circuit and era

5. Preprocessing experiments (>= 2)

On validation PR-AUC we compared: (1) impute-then-scale vs scale-then-impute, (2) median vs mean imputation, (3) excluding vs including Indianapolis 500 rounds. Results are in notebook section 7.

---

6. Modeling (>= 3 attempts)

Models: (1) logistic regression baseline, (2) random forest (200 trees, class weights), (3) shallow feed-forward neural network (64-32, dropout). Metrics from the successful Kaggle run (`acl-ms1`, saved version 2, Oct 19, 2026):

PR-AUC (primary):

Model	Train	Val	Test
Logistic regression	0.546	0.707	0.669
Random forest (default)	0.998	0.696	0.675
Shallow FFNN	0.580	0.724	0.664

Default random forest **overfits** (train PR-AUC ~0.99). A **regularized** forest (`max_depth=12`, larger `min_samples_leaf`, `max_samples=0.7`) yields train 0.630, val 0.718, **test 0.694**, reducing the train-val gap. See notebook section 8b.

Interpretation. All models beat a naive baseline on PR-AUC; test performance is similar across methods (~0.66–0.69), reflecting hard-to-predict race-day uncertainty. We report **test** metrics once, after choosing thresholds on validation.

---

7. Feature ablation

Dropping feature groups (logistic baseline) on validation/test PR-AUC is in notebook section 9 (`feature_ablation.csv`). Grid removal hurts test PR-AUC most, consistent with grid being a strong pre-race signal.

---

8. Explainability (XAI)

We report permutation importance, SHAP (global), and LIME (one validation row). **Disclaimer:** these show what the model relied on for predictions, not causal effects on real-world podiums.

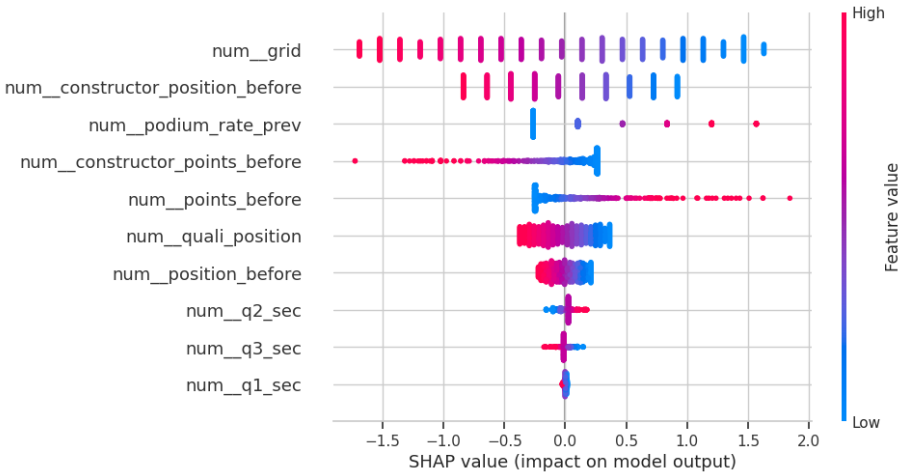


Figure 6: SHAP global feature importance

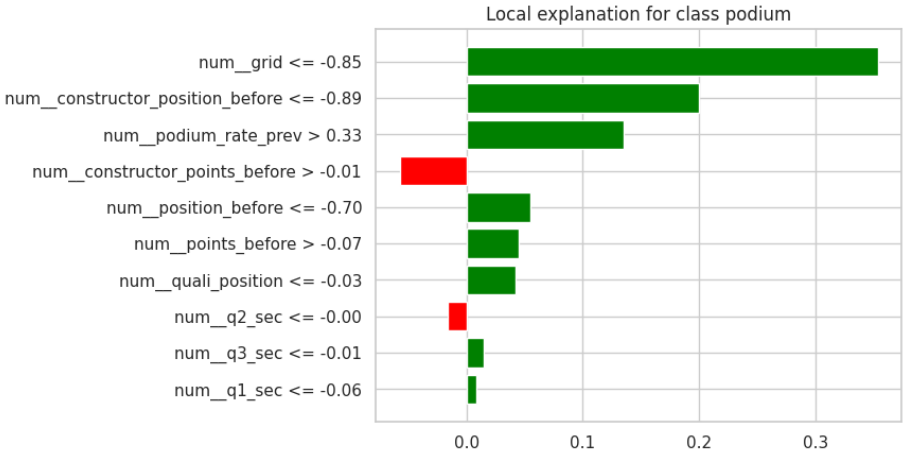


Figure 7: LIME explanation for one validation example

9. Limitations

- Qualifying features missing for many early seasons in Ergast.
- Rule and car changes across eras (temporal test set 2022+).
- Constructor renames and `constructorId` changes.
- Sprint weekends not modeled with sprint-specific features.
- Class imbalance; accuracy is misleading vs PR-AUC.

---

10. Inference

`src/inference.py` returns podium probability and a binary prediction for a single pre-race row; notebook section 11 demonstrates a complete row and a partial row with imputation.

References

- Ergast / Kaggle F1 CSV documentation
- Course Milestone 1 specification and newcomer materials
- Repository README and `GETTING_STARTED.md`